

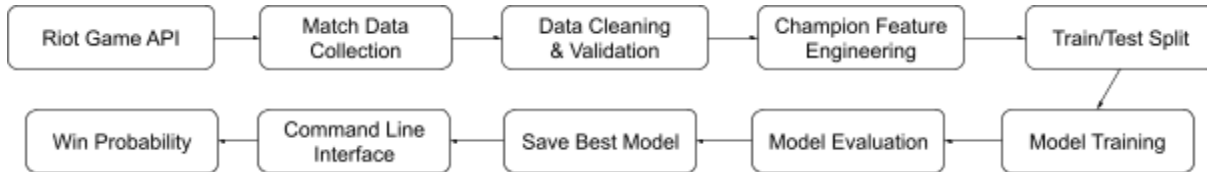
RiftPredict: Step-by-step Technical Guide

**League of Legends Draft Prediction Using Riot API Data and Machine Learning
Group 4**

1. Overview and System Architecture

This guide explains how to construct an end-to-end League of Legends draft prediction system that collects real ranked matches from the Riot Games API, cleans and transforms the data, engineers champion-composition features, trains several machine-learning classifiers, evaluates their performance, and connects the trained model to a command-line prediction interface.

Figure 1: System Pipeline



2. Environment and Project Setup

2.1. Prerequisites

This project was developed in Python and runs locally. Before running the system, the user should have Python 3 installed, Git if cloning the repository, and a Riot Games development API key if they want to collect new match data.

2.2. Project Setup

First, create and activate a virtual environment if necessary:

On macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

On Windows:

```
py -m venv .venv
.venv\Scripts\activate
```

Then, install the required Python libraries for this project, included in *requirements.txt*:

```
pip install -r requirements.txt
```

Riot Games requires an API key for authenticated requests. If you need to collect new match data, obtain a development API key from the Riot Games Developer Portal. Then, in the project root directory, create a file named *.env* and add the API key in the following format:

```
RIOT_API_KEY=RGAPI-your-key-here
```

Riot development API keys may expire. If requests begin returning authentication errors such as HTTP 401 or 403, generate a new key, replace the value in *.env*, and restart the program. Because *.env* contains a private API credential, it should be included in *.gitignore* and should not be committed.

3. Step 1: Data Collection

To run the data crawler from project root directory on macOS/Linux, enter:

```
python3 main.py \
  --game-name "MunchyPunchyLOL" \
  --tag-line "TTV1" \
  --target target_num \
  --matches-per-player match_per_player \
  --queue 420
```

On Windows, change *python3* to *py*.

Where *target_num* and *match_per_player* are replaced with the desired number.

3.1. Converting Riot ID to PUUID

Riot's Match-V5 API identifies players using a **PUUID**, rather than their visible Riot ID. Therefore, the collection pipeline first sends the player's game name and tag line to the Riot Account API. The returned PUUID is then used to retrieve the player's match history.

3.2. Retrieving and Processing Matches

The PUUID is passed to Riot's Match-V5 endpoint to obtain recent match IDs. The project specifies queue *ID 420* so that only Ranked Solo/Duo matches are requested. Each returned match ID is then used to retrieve the complete match record.

From each match, RiftPredict extracts the match ID, queue type, game version, game duration, winning team, and the five champions selected by each team. Champion selections are stored according to their assigned positions: Top, Jungle, Mid, ADC, and Support. The resulting dataset therefore contains records in the following form:

```
match_id | queue_id | winner | blue_top | blue_jg | blue_mid | blue_adc |  
blue_sup | red_top | red_jg | red_mid | red_adc | red_sup
```

Before a match is accepted, the collector verifies that it belongs to the requested queue, contains a valid winner, and has all ten required champion-role selections.

3.3. Expanding the Dataset

Collecting matches from only one player would produce a limited dataset. To expand the search automatically, the crawler extracts the PUUIDs of all ten participants from every downloaded match and adds previously unseen players to a queue. A set of previously seen match IDs is maintained to prevent duplicate records from being added to the dataset.

3.4. Rate Limits and Checkpointing

Riot limits the number of API requests that can be made within a short period. If the server returns *HTTP 429*, the program reads Riot's *Retry-After* value, waits for the requested duration, and retries automatically. Temporary server errors are also retried, while *HTTP 401* or *403* is treated as an invalid or expired API key.

Because collecting a large dataset can take a long time, the crawler periodically saves completed matches to *data/raw_matches.csv*. This checkpoint allows collection to resume after an interruption instead of discarding previously collected data.

Figure 2: Riot API Crawler Collecting Match Data

```
Checkpoint saved: 23550 total matches  
Downloading NA1_5593663692 (23550/25000 valid)  
Downloading NA1_5593653656 (23551/25000 valid)  
Downloading NA1_5593628659 (23552/25000 valid)  
Downloading NA1_5593618255 (23553/25000 valid)  
Downloading NA1_5593151219 (23554/25000 valid)  
Network error: HTTPConnectionPool(host='americas.api.riotg  
ames.com', port=443): Read timed out. (read timeout=20)  
Retrying in 1 seconds...  
Downloading NA1_5591876188 (23555/25000 valid)
```

4. Step 2: Preprocessing and Feature Engineering

After match collection, the raw Riot API data must be cleaned and transformed into numerical features before it can be used by the machine-learning models. Each raw match initially contains

champion names, role assignments, match metadata, and the winning team. The preprocessing pipeline converts this information into a consistent numerical representation of the two team compositions.

4.1. Preprocessing Data

The preprocessing script first loads the collected matches from *raw_matches.csv*. Duplicate records are removed using the unique *match_id*, preventing the same game from contributing more than once to model training. Matches shorter than 300 seconds are also excluded because these games are treated as likely remakes and may not represent a meaningful relationship between draft composition and final outcome.

Champion names must also be standardized before they can be matched with the champion metadata stored in *champions.json*. The preprocessing code normalizes capitalization and punctuation so that variations such as *Kai'Sa* and *Kaisa* can be matched consistently. It also handles Riot-specific naming differences, such as mapping *Wukong* to the internal identifier *MonkeyKing*.

The metadata file is then loaded and used to retrieve attributes for each champion, such as class tags and combat statistics.

4.2. Training/Test Split and Historical Win Rates

Before outcome-dependent features are calculated, the cleaned match dataset is split into 80% training data and 20% testing data. This ordering is important because the test set should remain unseen while training features are being created.

One feature used by RiftPredict is each champion's historical win rate. These win rates are calculated using only the training matches, preventing test-set outcomes from leaking into the model. The implementation also applies smoothing toward the overall win rate. This reduces unreliable extreme values for champions that appear only a small number of times in the training dataset.

4.3. Team Composition Features

Each team's five champions are kept in a fixed order:

Top → Jungle → Mid → ADC → Support

Maintaining a fixed role order allows role-dependent features, such as champion role frequency and lane matchup advantage, to be calculated consistently.

For every team, the feature-engineering stage calculates several categories of numerical features. These include:

- counts of Fighters, Tanks, Mages, Assassins, Marksmen, and Supports;
- number of ranged and melee champions;
- average attack, defense, magic, difficulty, HP, armor, and attack damage;
- average historical champion win rate;
- average AP/magic-damage ratio and its variance; and
- average role frequency: how commonly each champion is played in its assigned position.

Individual champion values are accumulated across the five picks and the continuous statistics are averaged to describe the overall composition of the team.

In addition to team-level statistics, RiftPredict also includes **lane matchup features**. The Blue and Red champions occupying the same lane are compared using a precomputed counter matrix. Positive values represent an advantage for Blue, while negative values favor Red. The current implementation includes matchup values for Top, Mid, and Support, which are the roles available in the counter dataset.

4.4. Constructing the Final Feature Vector

For every match, the preprocessing script first constructs the Blue and Red team lists from their role-specific champion columns. These lists are passed into the feature builder, which generates numerical composition features for each side and combines them with the cross-team lane matchup features.

The feature builder prefixes the team-specific values with *blue_* and *red_*, while lane-advantage values are added as shared cross-team features. The match winner is then encoded as the target variable, with *1* representing a Blue-team victory and *0* representing a Red-team victory.

5. Model Training and Evaluation

The training process can be run with:

On macOS/Linux:

```
python3 -m src.models.train_and_evaluate
```

On Windows, replace *python3* with *py*.

5.1. Training the Classifiers

After preprocessing, each match is represented by a numerical feature vector and a binary outcome label. The model inputs are stored in *X_train* and *X_test*, while *y_train* and *y_test* contain the corresponding match outcomes, where *1* represents a Blue-team win, while *0* represents a Red-team win.

RiftPredict trains and compares four supervised classification approaches: K-Nearest Neighbors (KNN), Decision Tree, Random Forest, and a soft-voting Ensemble. All four models are trained using the same feature set and training matches, allowing their performance to be compared under the same conditions.

The KNN classifier predicts an outcome based on similar historical match feature vectors. The Decision Tree learns a sequence of decision rules from the engineered features, while the Random Forest combines predictions from multiple decision trees. The Ensemble combines KNN, Decision Tree, and Random Forest using soft voting, which averages the probability estimates produced by the three classifiers.

The script trains each classifier and saves the models to the *saved_models* directory using Joblib.

5.2. Evaluating Model Performance

After training, each classifier is evaluated using the held-out test dataset. The test matches are not used to fit the models, allowing their predictions to be compared against previously unseen outcomes.

The evaluation script calculates four metrics for every classifier: accuracy, precision, recall, and weighted F1-score.

- **Accuracy** measures the percentage of test matches whose winner was predicted correctly.
- **Precision** measures how reliable the model's predicted classes are.
- **Recall** measures how successfully the model identifies the actual outcomes.
- **Weighted F1-score** combines precision and recall while accounting for the number of samples in each class.

Using several metrics provides more information than accuracy alone, especially if the numbers of Blue and Red victories are not perfectly balanced.

A confusion matrix is also produced for each classifier. It separates predictions according to the true and predicted class, making it possible to see whether a model is more successful at predicting one side than the other.

5.3. Model Comparison

The evaluation results for all four classifiers are saved to *results/metrics.csv*. Their test accuracies are also visualized in *model_comparison.png*, which provides a direct comparison between KNN, Decision Tree, Random Forest, and the Ensemble.

The Random Forest model is additionally used to generate a **feature-importance plot**, showing which engineered variables contributed most strongly to its decisions. This provides some insight into which aspects of team composition the model relies on most.

These results are used to evaluate the effectiveness of the trained models before connecting the selected model to the user-facing prediction interface.

6. Prediction Interface

Run this command from the project root after model training has been completed:

On macOS/Linux:

```
python3 -m src.ui.user_interface
```

On Windows, replace *python3* with *py*.

After the models have been trained, RiftPredict provides a command-line interface for evaluating a hypothetical League of Legends draft. The user enters five champions for the Blue team and five for the Red team in the fixed role order of Top, Jungle, Mid, ADC, and Support.

The entered champion selections are processed using the same feature-building procedure used during training. This ensures that live drafts use the same feature representation and ordering as the data used to train the classifier. Training-derived champion win rates are also saved and reused by the interface rather than recalculated from new data.

In addition to the numerical prediction, the interface provides composition notes describing potential strengths and weaknesses, such as frontline strength, damage balance, off-role picks, and lane matchup advantages.

Figure 3: User Interface

```
=====
                        RIFTPREDICT
=====
Please enter the full draft:

Blue Team Top: Chogath
Blue Team Jungle: Zac
Blue Team Mid: Vex
Blue Team Adc: Samira
Blue Team Support: Alistar

Red Team Top: Vayne
Red Team Jungle: Sejuani
Red Team Mid: Yasuo
Red Team Adc: Lux
Red Team Support: Sona

Predicted Winner: RED TEAM (70.7% confidence)

● Red Team Strengths: ['Balanced mixed damage (AD/AP)', 'High pick potential (assassin-heavy)']
● Blue Team Weaknesses: ['Heavy Magic Damage (easy for enemies to itemize MR)']
✂ Even matchup in: Top, Mid, Support

📊 Win Chances:
  • Blue Team: 29.3%
  • Red Team: 70.7%
=====
```

7. Troubleshooting

Table 1: Troubleshooting Table

Problem	Likely Cause	Solution
<i>ModuleNotFoundError</i>	Required Python package is missing or virtual environment is inactive	Activate <i>.venv</i> and run <i>python -m pip install -r requirements.txt</i>
Riot API returns 401/403	Development API key is invalid or expired	Generate a new key and replace <i>RIOT_API_KEY</i> in <i>.env</i>
Trained model cannot be loaded	Training has not been run or model file is missing	Run the model training script before launching the interface